

1) VHDL CODE OF TASK 1 HALF ADDERS

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity Half_Adder_AY is
5  port (
6      A : in std_logic;
7      B : in std_logic;
8      S : out std_logic;
9      C : out std_logic
10 );
11 end;
12
13 architecture data_flow of Half_Adder_AY is
14 begin
15
16     Sum: process (A,B) -- for S
17     begin
18         S <= A xor B;
19     end process;
20
21     Carry: process(A,B) -- for C
22     begin
23         C <= A and B;
24     end process;
25
26 end;
```

2) VHDL FOR TASK 2 FULL ADDER

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity Full_Adder_AY is
5  port(
6      A:in std_logic;
7      B:in std_logic;
8      Cin: in std_logic;
9      S: out std_logic;
10     Cout: out std_logic
11 );
12 end;
13
14 architecture Behavioral of Full_Adder_AY is
15     component Half_Adder_AY is
16     port(
17         A:in std_logic;
18         B:in std_logic;
19         C: out std_logic;
20         S: out std_logic
21     );
22     end component;
23     signal S1: std_logic; -- for 1st sum as A in 2nd half adder
24     signal C1: std_logic; -- carry 1 will be used for cout
25     signal C2: std_logic; -- carry 2 will also be used for cout
26 begin
27     half_adder1: Half_Adder_AY port map(
28         A => A,
29         B => B,
30         S => S1,
31         C => C1
32     );
33
34     half_adder2: Half_Adder_AY port map(
35         A => S1,
36         B => Cin,
37         S => S,
38         C => C2
39     );
40     Cout <= C1 or C2;
41 end;
42
```

3) VHDL CODE FOR TASK 3 4 Bit ADDER

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  library work;
5  use work.Yunayev_Adder_Package.all;
6
7
8  entity Bit4Full_Adder_AY is
9  generic(
10     constant N: integer := 4 -- N can now be change was not oringally;
11 );
12 port(
13     A:in std_logic_vector(N-1 downto 0);
14     B:in std_logic_vector(N-1 downto 0);
15     Cin: in std_logic;
16     S: out std_logic_vector(N-1 downto 0);
17     Cout: out std_logic
18 );
19 end;
20
21
22
23 architecture structural of Bit4Full_Adder_AY is
24
25     signal CarryOuts:std_logic_vector(N downto 0);
26 begin
27     CarryOuts(0) <= Cin;
28     forloop: for i in 0 to N-1 generate
29         fourBit_adder: Full_Adder_AY port map(
30             A =>A(i),
31             B => B(i),
32             Cin => CarryOuts(i),
33             S => S(i),
34             Cout => CarryOuts(i+1)
35         );
36     end generate forloop;
37     Cout <= CarryOuts(N);
38 end;
```

4) VHDL CODE FOR TASK 4 Bit ADD/SUB ALSO WILL BE USED AS N-BIT ADD/SUB

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  library work;
5  use work.Yunayev_Adder_Package.all;
6
7  entity BitNFull_Add_Sub_AY is
8  generic(
9      N : integer := 4 -- Define N as a generic
10 );
11 port(
12     A:in std_logic_vector(N-1 downto 0);
13     B:in std_logic_vector(N-1 downto 0);
14     Cin: in std_logic; -- will be the same as Operational code if 0 do add, if 1 do subtraciton
15     S: out std_logic_vector(N-1 downto 0);
16     Cout: out std_logic
17 );
18 end;
19
20
21
22 architecture Behavioral of BitNFull_Add_Sub_AY is
23     signal CarryOuts:std_logic_vector(N downto 0);
24     signal overflow: std_logic;
25     signal zero:std_logic;
26     signal negative:std_logic;
27     signal S_Signal:std_logic_vector(N-1 downto 0);
28     signal B_inverted:std_logic_vector(N-1 downto 0);
29
30
31 begin
32     CarryOuts(0) <= Cin;
33     forloop: for i in 0 to N-1 generate
34         B_inverted(i) <= B(i) xor Cin;
35         fourBit_add_sub: Full_Adder_AY port map(
36             A =>A(i),
37             B => B_inverted(i), --B(i) WILL BE OUR ERROR
38             Cin => CarryOuts(i),
```

```

39         S => S(i),
40         Cout => CarryOuts(i+1)
41     );
42     S_Signal(i) <= A(i) xor (B(i) xor Cin) xor CarryOuts(i);
43
44     end generate forloop;
45
46     Cout <= CarryOuts(N);
47     overflow <= CarryOuts(N) xor CarryOuts(N-1);
48     negative <= S_Signal(N-1);
49
50
51     process(S_Signal)
52     variable all_zero : boolean := true;
53     begin
54         for i in 0 to N-1 loop
55             if S_Signal(i) = '1' then
56                 all_zero := false;
57                 exit;
58             end if;
59         end loop;
60
61         if all_zero then
62             zero <= '1';
63         else
64             zero <= '0';
65         end if;
66     end process;
67
68
69     process(overflow, zero, negative)
70     begin
71
72         if overflow = '1' then
73             report "There is an overflow" severity warning;
74         end if;
75
76         if zero = '1' then
77             report "The Sum is zero" severity warning;
78         end if;
79
80         if negative = '1' then
81             report "The sum is negative" severity warning;
82         end if;
83     end process;
84
85 end;

```

5) VHDL CODE FOR TASK 5 PACKAGE COMPONENTS

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4
5  package Yunayev_Adder_Package is
6  | constant N: integer := 4;
7  | component Half_Adder_AY is
8  |   port (
9  |     A : in std_logic;
10 |     B : in std_logic;
11 |     S : out std_logic;
12 |     C : out std_logic
13 |   );
14 | end component;
15
16 | component Full_Adder_AY is
17 |   port(
18 |     A:in std_logic;
19 |     B:in std_logic;
20 |     Cin: in std_logic;
21 |     S: out std_logic;
22 |     Cout: out std_logic
23 |   );
24 | end component;
25
26 | component BitNFull_Add_Sub_AY is
27 |   port(
28 |
29 |     A:in std_logic_vector(N-1 downto 0);
30 |     B:in std_logic_vector(N-1 downto 0);
31 |     Cin: in std_logic;-- will be the same as Operational code if 0 do add, if 1 do subtraciton
32 |     S: out std_logic_vector(N-1 downto 0);
33 |     Cout: out std_logic
34 |   );
35 | end component;
36
37
38
39 | component LPM_ADD_SUB_AY
40 |   PORT
41 |   (
42 |     add_sub : IN STD_LOGIC ;
43 |     cin : IN STD_LOGIC ;
44 |     dataa : IN STD_LOGIC_VECTOR (0 DOWNT0 0);
45 |     datab : IN STD_LOGIC_VECTOR (0 DOWNT0 0);
46 |     cout : OUT STD_LOGIC ;
47 |     overflow : OUT STD_LOGIC ;
48 |     result : OUT STD_LOGIC_VECTOR (0 DOWNT0 0)
49 |   );
50 | end component;
51
52
53
54 end package;
```

6) VHDL CODE for TASK 6A) RIPPLE CARRY

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  library work;
5  use work.Yunayev_Adder_Package.all;
6
7  entity NbitRipple_Add_Sub_AY is
8  generic(
9      N : integer := 32 -- Define N as a generic
10 );
11 port(
12     A:in std_logic_vector(N-1 downto 0);
13     B:in std_logic_vector(N-1 downto 0);
14     Cin: in std_logic;-- will be the same as Operational code if 0 do add, if 1 do subtraciton
15     S: out std_logic_vector(N-1 downto 0);
16     Cout: out std_logic
17 );
18 end;
19
20 -- SIMILAR TO THE FULL NBIT ADD/SUB
21 architecture Behavioral of NbitRipple_Add_Sub_AY is
22
23     signal B_inverted:std_logic_vector(N-1 downto 0);
24     signal CarryOuts:std_logic_vector(N downto 0);
25     signal CarryOuts_delay:std_logic_vector(N downto 0);
26     signal S_delay: std_logic_vector(N-1 downto 0);
27     signal overflow: std_logic;
28     signal zero:std_logic;
29     signal negative:std_logic;
30
31 begin
32     CarryOuts_delay(0) <= Cin;
33     forloop: for i in 0 to N-1 generate
34         B_inverted(i) <= B(i) xor Cin;
35         NBit_adder: Full_Adder_AY port map(
36             A =>A(i),
37             B => B_inverted(i),
38             Cin => CarryOuts_delay(i),

```

```

39         S => S_delay(i) ,
40         Cout => CarryOuts_delay(i+1)
41     );
42     end generate forloop;
43
44     process(S_delay,CarryOuts_delay)
45     begin
46         for i in 0 to N-1 loop
47             S(i) <= S_delay(i) after 10 ps;
48
49             CarryOuts(i) <= CarryOuts_delay(i) after 10 ps;
50         end loop;
51         CarryOuts(N) <= CarryOuts_delay(N) after 10 ps;
52     end process;
53
54     Cout <= CarryOuts(N);
55     overflow <= CarryOuts(N) xor CarryOuts(N-1);
56     negative <= S_delay(N-1);
57
58
59     process(S_delay)
60     variable all_zero : boolean := true;
61     begin
62         for i in 0 to N-1 loop
63             if S_delay(i) = '1' then
64                 all_zero := false;
65                 exit;
66             end if;
67         end loop;
68
69         if all_zero then
70             zero <= '1';
71         else
72             zero <= '0';
73         end if;
74     end process;
75
76
77     process(overflow, zero, negative)
78     begin
79
80         if overflow = '1' then
81             report "There is an overflow" severity warning;
82         end if;
83
84         if zero = '1' then
85             report "The Sum is zero" severity warning;
86         end if;
87
88         if negative = '1' then
89             report "The sum is negative" severity warning;
90         end if;
91     end process;
92
93
94 end;

```


7) VHDL CODE FOR TASK 6B) CARRY LOOK AHEAD

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  library work;
5  use work.Yunayev_Adder_Package.all;
6
7  entity NbitCarryLook_Add_Sub_AY is
8  generic(
9      N : integer := 32
10 );
11 port(
12
13     A:in std_logic_vector(N-1 downto 0);
14     B:in std_logic_vector(N-1 downto 0);
15     Cin: in std_logic;-- will be the same as Operational code if 0 do add, if 1 do subtraciton
16     S: out std_logic_vector(N-1 downto 0);-- for carry out
17     Cout: out std_logic
18 );
19 end;
20
21 -- SIMILAR TO THE FULL NBIT ADD/SUB
22 architecture structural of NbitCarryLook_Add_Sub_AY is
23     signal B_inverted:std_logic_vector(N-1 downto 0);
24     signal S_Signal:std_logic_vector(N-1 downto 0);
25     signal CarryIns:std_logic_vector(N downto 0);
26     signal P: std_logic_vector(N-1 downto 0);
27     signal G: std_logic_vector(N-1 downto 0);
28
29     signal overflow: std_logic;
30     signal zero:std_logic;
31     signal negative:std_logic;
32
33 begin
34     CarryIns(0) <= Cin;
35     forloop: for i in 0 to N-1 generate
36     B_inverted(i) <= B(i) xor Cin;
37     NBit_add_Sub: Full_Adder_AY port map(
38         A =>A(i),
39         B => B_inverted(i),
40         Cin => CarryIns(i),
41         S => open,--S_signal(i) had a weird error so did the solution manually
42         Cout => open --will use later
43     );
44     end generate forloop;
45
46     looker:for i in 0 to N-1 generate
47     P(i) <= A(i) xor B_inverted(i); -- For Carry Looks
48     G(i) <= A(i) and B_inverted(i); -- For Carry Looks
49     CarryIns(i+1) <= G(i) or (P(i) and CarryIns(i));
50     S_signal(i) <= P(i) xor CarryIns(i);
51     end generate looker;
52
53     S <= S_signal;
54     Cout <= CarryIns(N);
55
56     overflow <= CarryIns(N) xor CarryIns(N-1);
57     negative <= S_Signal(N-1);
58
59     process(S_Signal)
60     variable all_zero : boolean := true;
61     begin
62         for i in 0 to N-1 loop
63             if S_Signal(i) = '1' then
64                 all_zero := false;
65                 exit;
66             end if;
67         end loop;
68
69         if all_zero then
70             zero <= '1';
71         else
72             zero <= '0';
73         end if;
74     end process;
75
76 end structural;

```

```

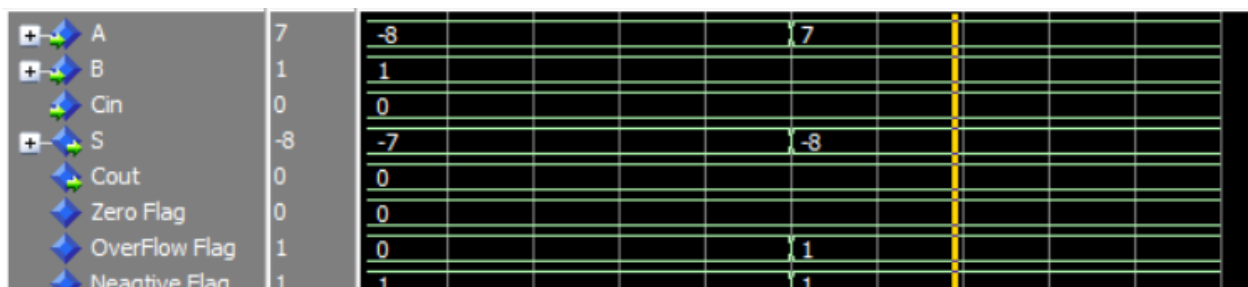
77         end if;
78     end process;
79
80
81     process(overflow, zero, negative)
82     begin
83
84         if overflow = '1' then
85             report "There is an overflow" severity warning;
86         end if;
87
88         if zero = '1' then
89             report "The Sum is zero" severity warning;
90         end if;
91
92         if negative = '1' then
93             report "The sum is negative" severity warning;
94         end if;
95     end process;
96 end;
97
98
99

```

WaveForms

N=4 ADD/SUB

a. Most Positive N bit integer + 1



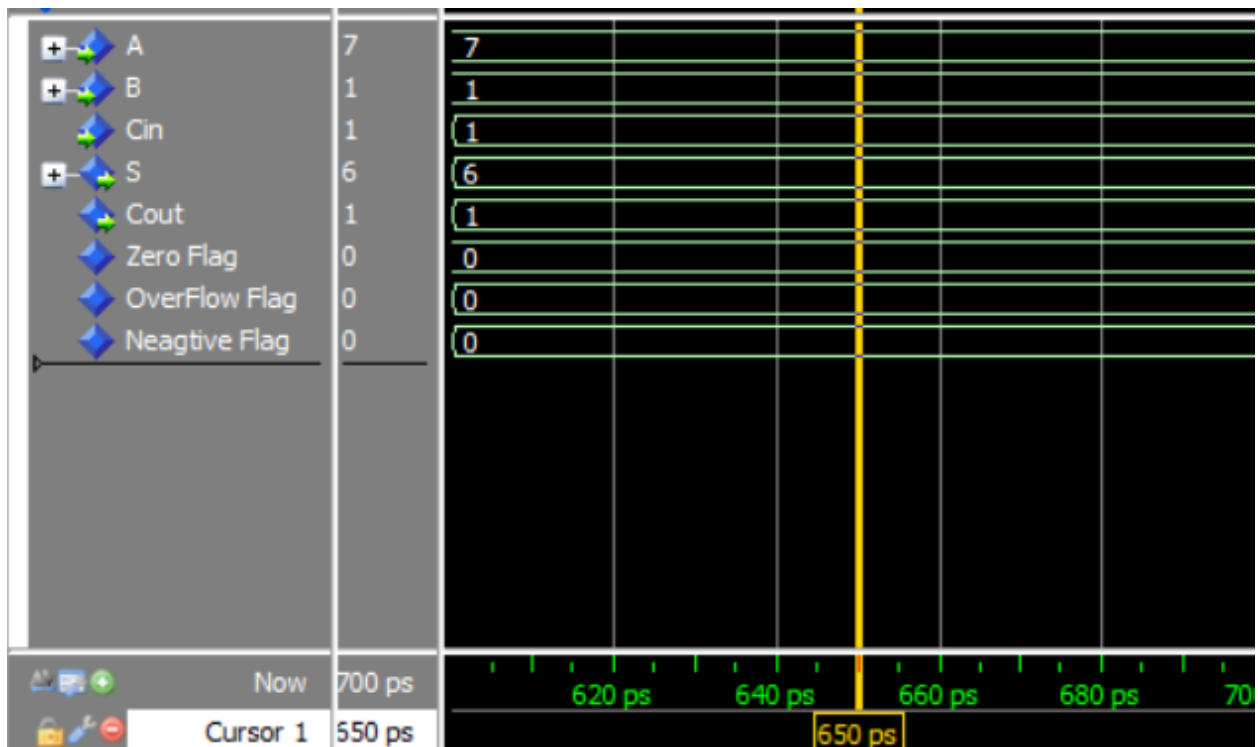
VSIM 30> run

```

# ** Warning: There is an overflow
#   Time: 500 ps  Iteration: 7  Instance: /bitnfull_add_sub_ay
# ** Warning: The sum is negative
#   Time: 500 ps  Iteration: 8  Instance: /bitnfull_add_sub_ay

```

b. Most Positive N bit integer – 1

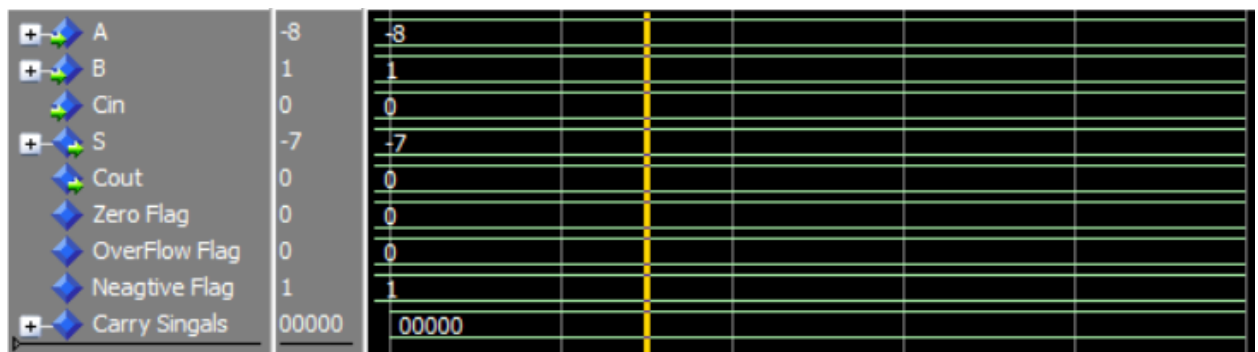


```

force -freeze sim:/bitnfull_add_sub_ay/Cin 1 0
VSIM 32> run
VSIM 33>

```

c. Most Negative N bit integer + 1



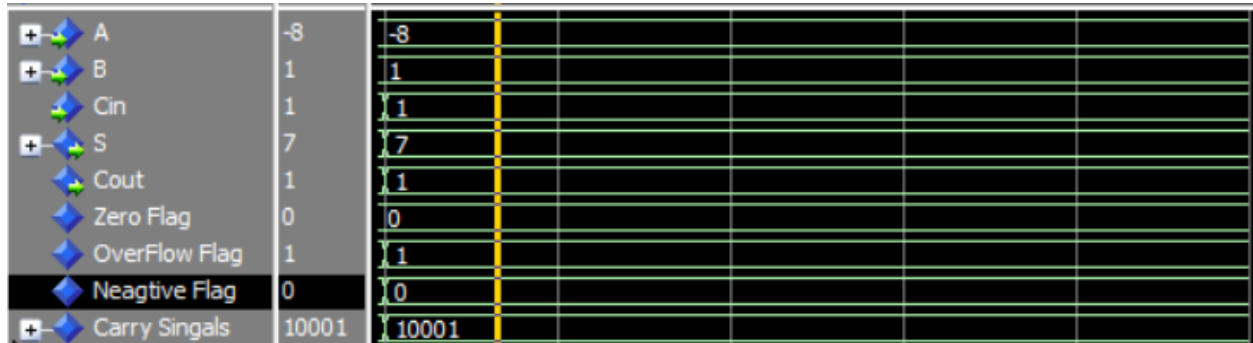
```

# ** Warning: The sum is negative
#   Time: 700 ps  Iteration: 6  Instance: /bitnfull_add_sub_ay
# ** Warning: There is an overflow
#   Time: 700 ps  Iteration: 6  Instance: /bitnfull_add_sub_ay

```

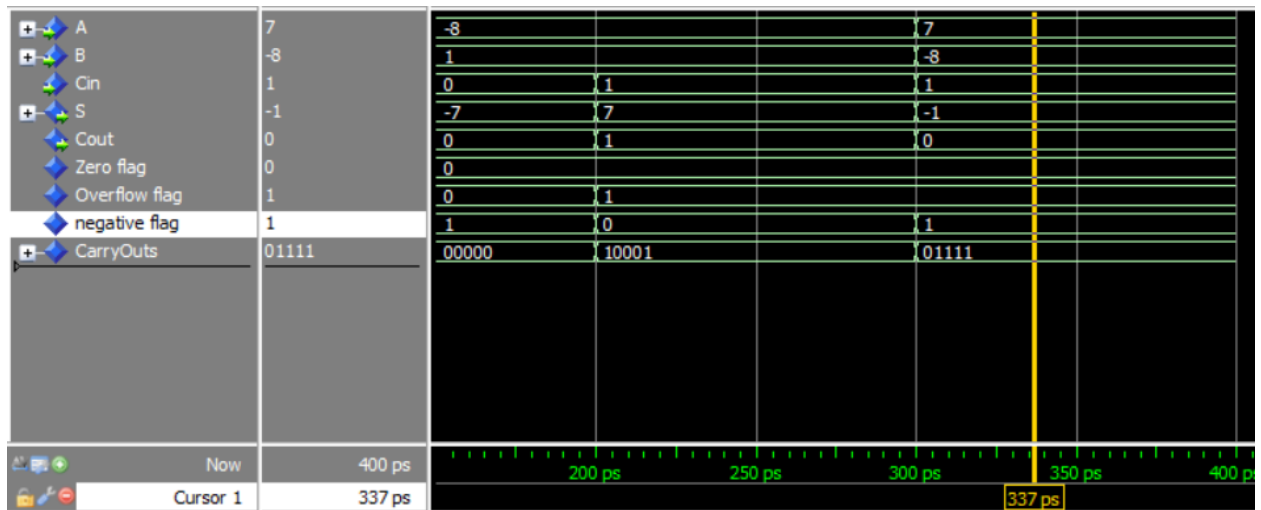
For some reason the assertion text overflow flag turns on even though Overflow flow is off and the carry signals are also 0 so this is a odd glitch.

d. Most Negative N bit integer - 1



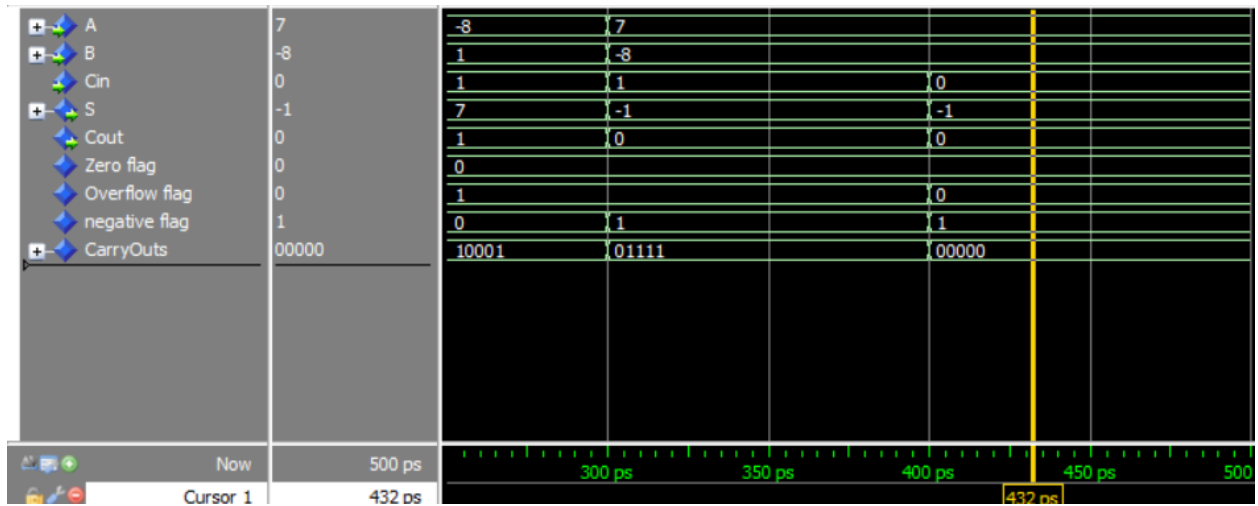
Flags warning text not working properly unknown reason

e. Most Positive N bit integer- Most Negative N bit integer



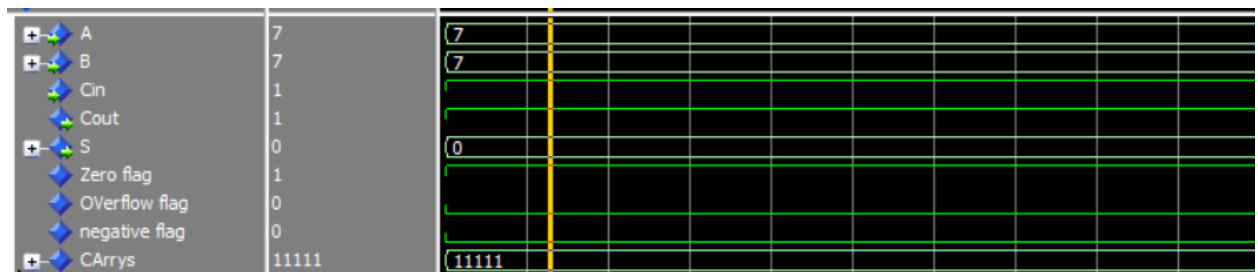
```
# ** Warning: There is an overflow
#   Time: 300 ps  Iteration: 4  Instance: /bitnfull_add_sub_ay
# ** Warning: The sum is negative
#   Time: 300 ps  Iteration: 4  Instance: /bitnfull_add_sub_ay
```

f. Most Positive N bit integer+ Most Negative N bit integer



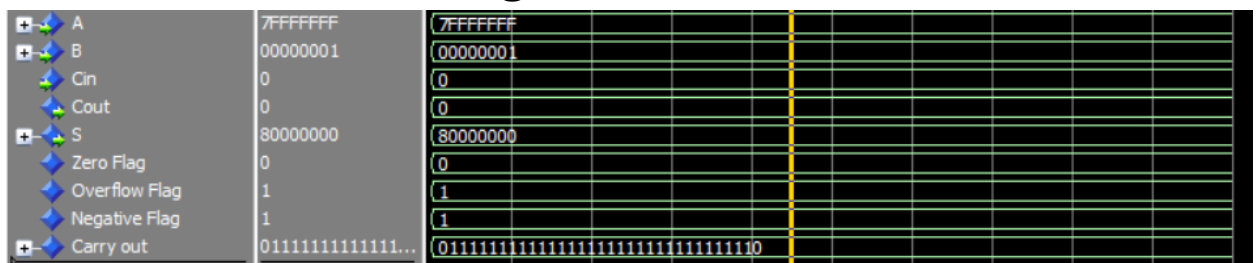
```
# ** Warning: The sum is negative
#   Time: 400 ps  Iteration: 10  Instance: /bitnfull_add_sub_ay
VSIM 28> ]
```

g. Most Positive N bit integer- Most Positive N bit integer



N=32 BIT ADD/SUB

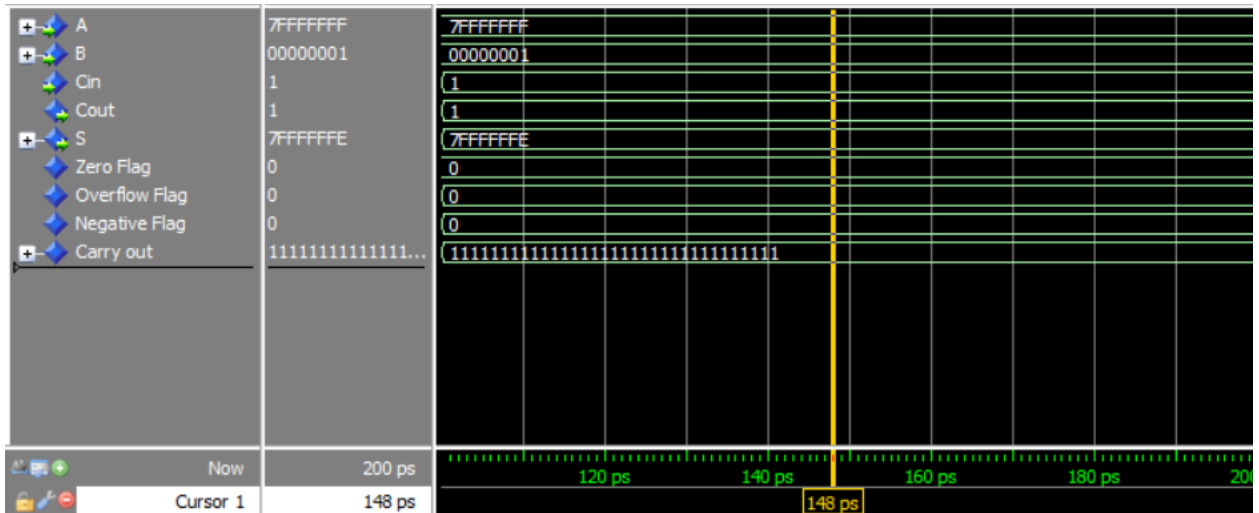
a. Most Positive N bit integer + 1



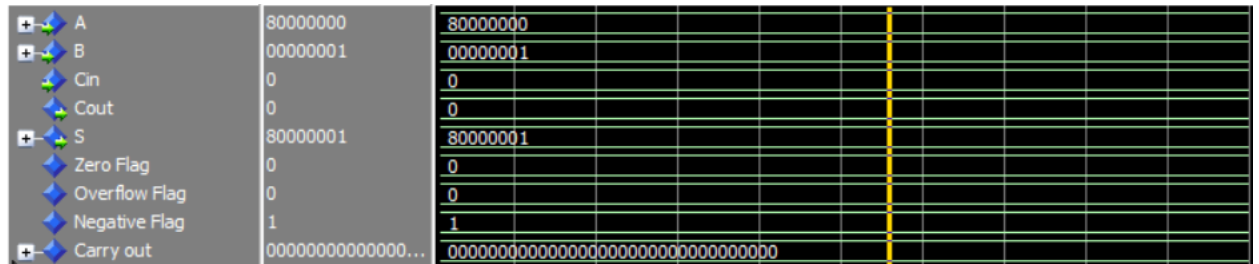
```
# ** Warning: There is an overflow
#   Time: 0 ps   Iteration: 65   Instance: /bitnfull_add_sub_ay
# ** Warning: The sum is negative
#   Time: 0 ps   Iteration: 65   Instance: /bitnfull_add_sub_ay
```

VSIM 18>

b. Most Positive N bit integer - 1



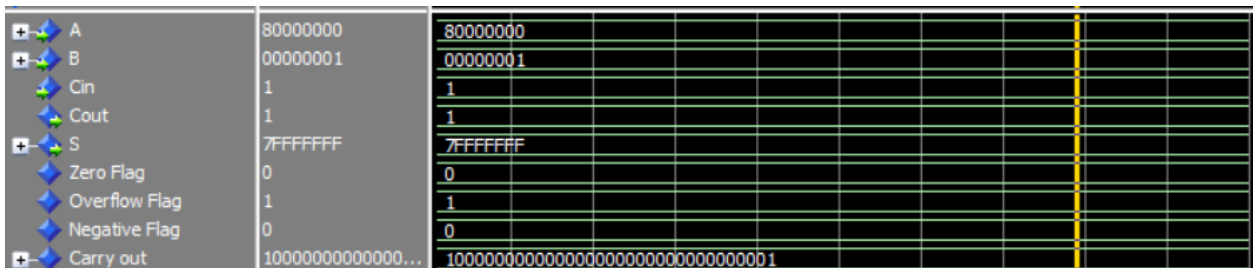
c. Most Negative N bit integer + 1

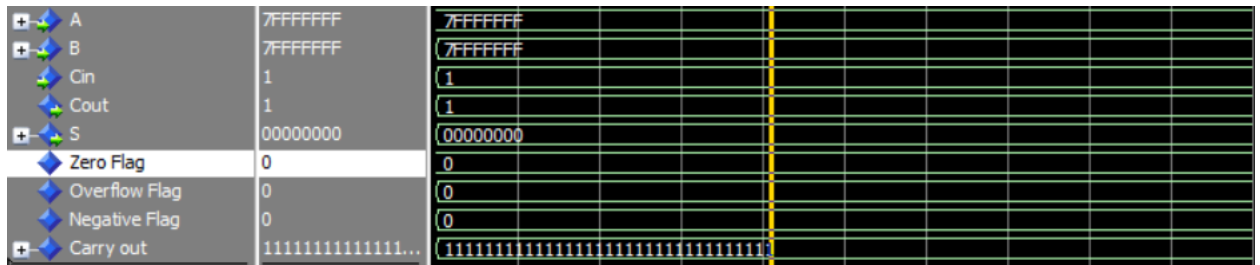


```
# ** Warning: The sum is negative
#   Time: 200 ps   Iteration: 7   Instance: /bitnfull_add_sub_ay
```

/SIM 23>

d. Most Negative N bit integer - 1

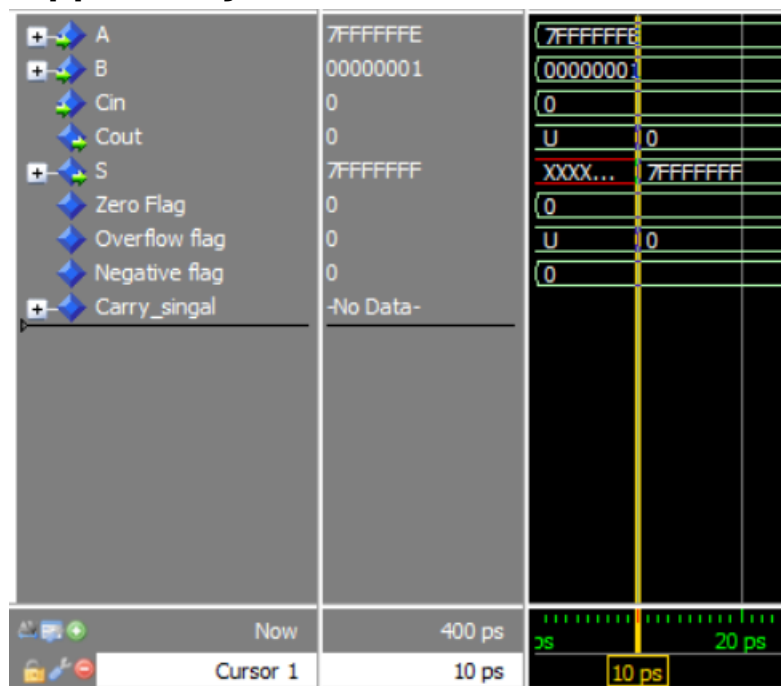




Zero Flag is not up this is wrong maybe zero flag does not get reset so its stuck

a. Measure time your design 6A, and 6B) adds two 32-bit numbers using ripple –carry and carry-look-ahead

Ripple carry



Look ahead

	msgs					
A	7FFFFFFE	7FFFFFFE				
B	00000001	00000001				
Cin	0	0				
Cout	0	0				
S	7FFFFFFF	7FFFFFFF				

N=4 for LPM

a. Most Positive N bit integer + 1

A	7	7				
B	1	1				
Cin	1	1				
Cout	0	0				
S	8	8				
Zero Flag	0	0				
Overflow	1	1				
Negative	1	1				

```
# ** Warning: There is an overflow
#   Time: 0 ps  Iteration: 10  Instance: /bitn_lpm_add_sub_ay
# ** Warning: The sum is negative
#   Time: 0 ps  Iteration: 10  Instance: /bitn_lpm_add_sub_ay
```

b. Most Positive N bit integer - 1

A	7	7				
B	1	1				
Cin	0	0				
Cout	1	1				
S	6	6				
Zero Flag	0	0				
Overflow	0	0				
Negative	0	0				

c. Most Negative N bit integer + 1

A	8	8
B	1	1
Cin	1	1
Cout	0	0
S	9	9
Zero Flag	0	0
Overflow	0	0
Negative	1	1

```
# ** Warning: The sum is negative
#   Time: 200 ps  Iteration: 6  Instance: /bitn_lpm_add_sub_ay
```

d. Most Negative N bit integer – 1

A	8	8
B	1	1
Cin	0	0
Cout	1	1
S	7	7
Zero Flag	0	0
Overflow	1	1
Negative	0	0

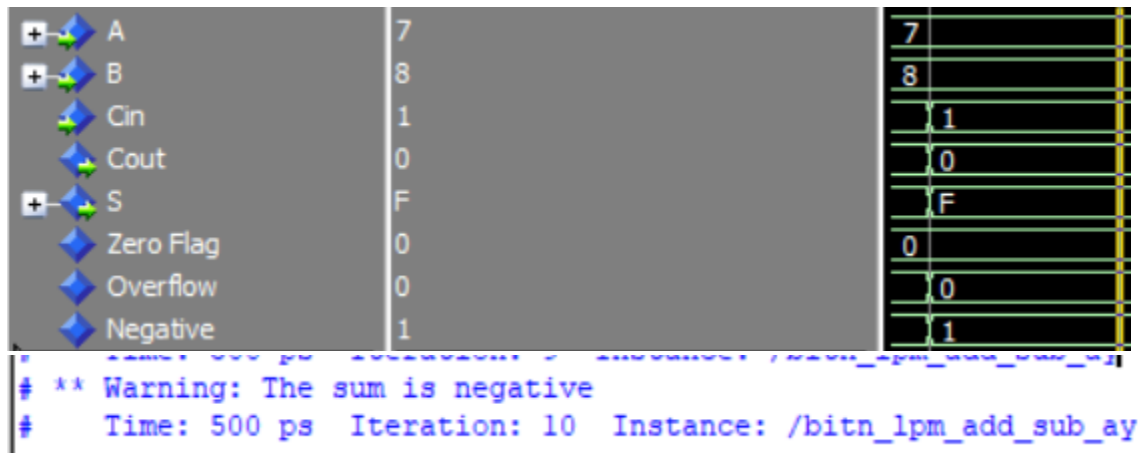
```
VSim 21> Run
# ** Warning: There is an overflow
#   Time: 300 ps  Iteration: 3  Instance: /bitn_lpm_add_sub_ay
```

e. Most Positive N bit integer- Most Negative N bit integer

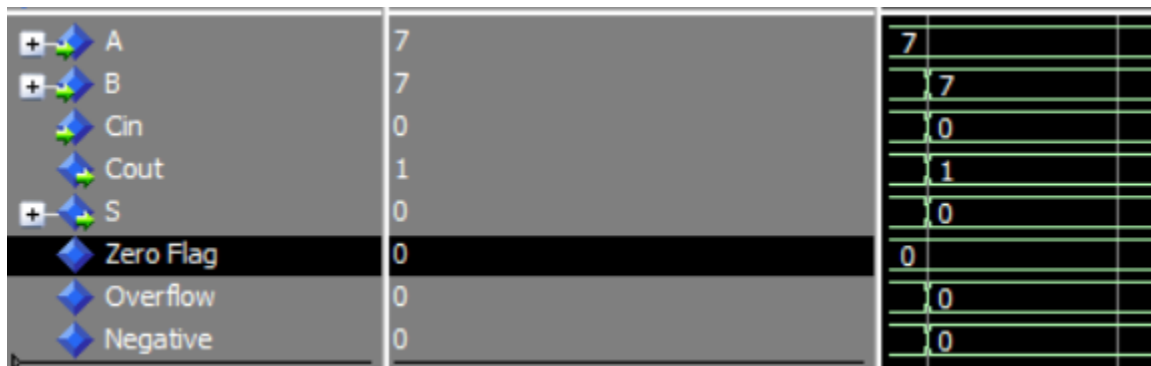
A	7	7
B	8	8
Cin	0	0
Cout	0	0
S	F	F
Zero Flag	0	0
Overflow	1	1
Negative	1	1

```
# ** Warning: There is an overflow
#   Time: 400 ps  Iteration: 5  Instance: /bitn_lpm_add_sub_ay
# ** Warning: The sum is negative
#   Time: 400 ps  Iteration: 5  Instance: /bitn_lpm_add_sub_ay
```

f. Most Positive N bit integer+ Most Negative N bit integer



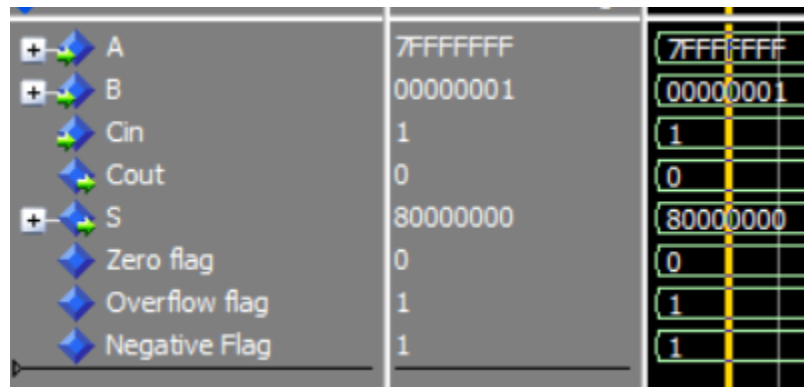
g. Most Positive N bit integer- Most Positive N bit integer



Zero flag does not work

N=32 LPM

a. Most Positive N bit integer + 1



```
# ** Warning: There is an overflow
#   Time: 0 ps   Iteration: 66   Instance: /bitn_lpm_add_sub_ay
# ** Warning: The sum is negative
#   Time: 0 ps   Iteration: 66   Instance: /bitn_lpm_add_sub_ay
```

b. Most Positive N bit integer - 1

	A	7FFFFFFF	7FFFFFFF
	B	00000001	00000001
	Cin	0	0
	Cout	1	1
	S	7FFFFFFE	7FFFFFFE
	Zero flag	0	0
	Overflow flag	0	0
	Negative Flag	0	0

c. Most Negative N bit integer + 1

	A	80000000	80000000
	B	00000001	00000001
	Cin	1	1
	Cout	0	0
	S	80000001	80000001
	Zero flag	0	0
	Overflow flag	0	0
	Negative Flag	1	1

```
# ** Warning: The sum is negative
#   Time: 200 ps   Iteration: 6   Instance: /bitn_lpm_add_sub_ay
```

d. Most Negative N bit integer - 1

	A	80000000	80000000
	B	00000001	00000001
	Cin	0	0
	Cout	1	1
	S	7FFFFFFF	7FFFFFFF
	Zero flag	0	0
	Overflow flag	1	1
	Negative Flag	0	0

```
** Warning: There is an overflow
   Time: 300 ps   Iteration: 3   Instance: /bitn_lpm_add_sub_ay
```

e. Most Positive N bit integer- Most Negative N bit integer

A	7FFFFFFF	7FFFFFFF
B	80000000	80000000
Cin	0	0
Cout	0	0
S	FFFFFFFF	FFFFFFFF
Zero flag	0	0
Overflow flag	1	1
Negative Flag	1	1

```
# ** Warning: There is an overflow
#   Time: 400 ps  Iteration: 5  Instance: /bitn_lpm_add_sub_ay
# ** Warning: The sum is negative
#   Time: 400 ps  Iteration: 5  Instance: /bitn_lpm_add_sub_ay
```

f. Most Positive N bit integer+ Most Negative N bit integer

A	7FFFFFFF	7FFFFFFF
B	80000000	80000000
Cin	1	1
Cout	0	0
S	FFFFFFFF	FFFFFFFF
Zero flag	0	0
Overflow flag	0	0
Negative Flag	1	1

```
# ** Warning: The sum is negative
#   Time: 500 ps  Iteration: 66  Instance: /bitn_lpm_add_sub_ay
```

g. Most Positive N bit integer- Most Positive N bit integer

A	7FFFFFFF	7FFFFFFF
B	7FFFFFFF	7FFFFFFF
Cin	0	0
Cout	1	1
S	00000000	00000000
Zero flag	0	0
Overflow flag	0	0
Negative Flag	0	0

Again the Zero flag is not showing

TB VHDL:

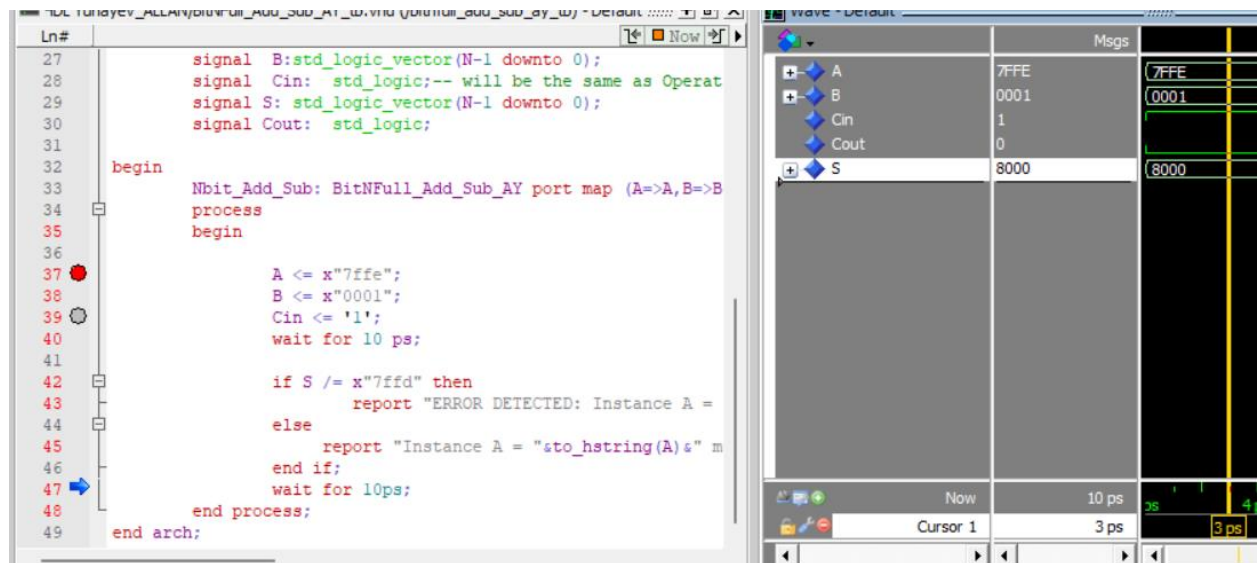
```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  library work;
5  use work.Yunayev_Adder_Package.all;
6
7  entity BitNFull_Add_Sub_AY_tb is
8  generic(
9      N: integer := 16
10 );
11 end entity;
12
13 architecture arch of BitNFull_Add_Sub_AY_tb is
14 component BitNFull_Add_Sub_AY is
15 port(
16
17     A:in std_logic_vector(N-1 downto 0);
18     B:in std_logic_vector(N-1 downto 0);
19     Cin: in std_logic;-- will be the same as Operational code if 0 do add, if 1 do subtraciton
20     S: out std_logic_vector(N-1 downto 0);
21     Cout: out std_logic
22
23 );
24 end component;
25
26 signal A: std_logic_vector(N-1 downto 0);
27 signal B:std_logic_vector(N-1 downto 0);
28 signal Cin: std_logic;-- will be the same as Operational code if 0 do add, if 1 do subtraciton
29 signal S: std_logic_vector(N-1 downto 0);
30 signal Cout: std_logic;
31
32 begin
33     Nbit_Add_Sub: BitNFull_Add_Sub_AY port map (A=>A,B=>B,Cin=>Cin,S=>S,Cout => Cout );
34     process
35     begin
36
37         A <= x"7ffe";
38         B <= x"0001";
39         Cin <= '1';
40         wait for 10 ps;
41
42         if S /= x"7ffd" then
43             report "ERROR DETECTED: Instance A = " & to_hstring(A)
44             & " minus Instance B = " & to_hstring(B) & "The sum should be 7FFD YOUR SUM IS " & to_hstring(S) severity warning;
45         else
46             report "Instance A = " & to_hstring(A) & " minus Instance B = " & to_hstring(B) & " Is " & to_hstring(S);
47         end if;
48         wait for 10ps;
49     end process;
50 end arch;
```

ERROR on ADD/SUB

$B \Rightarrow \bar{B}(i)$, --B_inverted(i), B(i) WILL BE OUR ERROR

Error

```
VSIM 75> step -over -current
# ** Warning: ERROR DETECTED: Instance A = 7FFE minus Instance B = 0001The sum should be 7FFD YOUR SUM IS 8000
# Time: 10 ps Iteration: 0 Instance: /bitnfull_add_sub_ay_tb
VSIM 75>
```



We mapped b(i) to b(i) instead b_inverted(i) to create the error.

No Error

B => B_inverted(i), --B(i) WILL BE OUR ERROR

```

signal A: std_logic_vector(N-1 downto 0);
signal B: std_logic_vector(N-1 downto 0);
signal Cin: std_logic; -- will be the same as Operat
signal S: std_logic_vector(N-1 downto 0);
signal Cout: std_logic;

begin

  Nbit_Add_Sub: BitNFull_Add_Sub_AY port map (A=>A,B=>B
process
begin

    A <= x"7ffe";
    B <= x"0001";
    Cin <= '1';
    wait for 10 ps;

    if S /= x"7ffd" then
        report "ERROR DETECTED: Instance A =
    else
        report "Instance A = " &to_hstring(A) & " m
    end if;
    wait for 10ps;
end process;

```

Signal	Value	Time
A	7FFE	7FFE
B	0001	0001
Cin	1	1
Cout	1	1
S	7FFD	7FFD

Now
10 ps

Cursor 1
5 ps

** Note: Instance A = 7FFE minus Instance B = 0001 Is 7FFD
Time: 10 ps Iteration: 0 Instance: /bitnfull_add_sub_ay_tb

/SIM 93>